

LAPORAN PRAKTIKUM JOBSHEET 12



Disusun Oleh:

HAFIZH ARRASYIID SYAHBANA

254107060026

SIB 1A

**PROGRAM STUDI D4 SISTEM INFORMASI
BISNIS JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI
MALANG 2026**

12.2.2 Verifikasi Hasil Percobaan

 Mahasiswa10.java src 1

```
1  public class Mahasiswa10 {      Mahasiswa10.java is a non-project file, only s
2      String nim;
3      String nama;
4      String kelas;      "kelas": Unknown word.
5      double ipk;
6
7      public Mahasiswa10(String nim, String nama, String kelas, double ipk) {
8          this.nim = nim;
9          this.nama = nama;
10         this.kelas = kelas;      "kelas": Unknown word.
11         this.ipk = ipk;
12     }
13
14     public void tampil() {      "tampil": Unknown word.
15         System.out.println(
16             "NIM    : " + nim +
17             "\nNama  : " + nama +
18             "\nKelas : " + kelas +      "Kelas": Unknown word.
19             "\nIPK   : " + ipk
20         );
21     }
22 }
```

 Node10.java src 1

```
1  public class Node10 {      Node10.java is
2      Mahasiswa10 data;      "Mahasiswa": U
3      Node10 prev;
4      Node10 next;
5
6      public Node10(Mahasiswa10 data) {
7          this.data = data;
8          this.prev = null;
9          this.next = null;
10     }
11 }
12
```

```

DoubleLinkedList10.java src 1
1  public class DoubleLinkedList10 {    DoubleLinkedList10.java is a non-project file,
2      Node10 head;
3      Node10 tail;
4
5      public DoubleLinkedList10() {
6          head = null;
7          tail = null;
8      }
9
10     public boolean isEmpty() {
11         return head == null;
12     }
13
14     public void addFirst(Mahasiswa10 data) {    "Mahasiswa": Unknown word.
15         Node10 newNode = new Node10(data);
16         if (isEmpty()) {
17             head = tail = newNode;
18         } else {
19             newNode.next = head;
20             head.prev = newNode;
21             head = newNode;
22         }
23     }
24
25     public void addLast(Mahasiswa10 data) {    "Mahasiswa": Unknown word.
26         Node10 newNode = new Node10(data);
27         if (isEmpty()) {
28             head = tail = newNode;
29         } else {
30             tail.next = newNode;
31             newNode.prev = tail;
32             tail = newNode;
33         }
34     }
35
36     public void insertAfter(String keyNim, Mahasiswa10 data) {    "Mahasiswa": Unkn
37         Node10 current = head;
38         while (current != null && !current.data.nim.equals(keyNim)) {
39             current = current.next;
40         }
41         if (current == null) {
42             System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
43             return;
44         }
45
46         Node10 newNode = new Node10(data);
47
48         if (current == tail) {
49             newNode.prev = current;
50             current.next = newNode;
51             tail = newNode;
52         } else {
53             newNode.prev = current;
54             newNode.next = current.next;
55             current.next.prev = newNode;
56             current.next = newNode;
57         }
58         System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);    "b
59     }
60
61     public void print() {
62         if (isEmpty()) {
63             System.out.println(x: "Linked List masih kosong.");    "masih": Unknown
64             return;
65         }
66         Node10 current = head;
67         while (current != null) {
68             current.data.tampil();
69             current = current.next;
70         }
71     }
72 }

```

DoubleLinkedListMain10.java src 3

```
1 import java.util.Scanner; DoubleLinkedListMain10.java is a non-project file, on
2
3 public class DoubleLinkedListMain10 {
4
5     static Mahasiswa10 inputMahasiswa(Scanner scan) { "Mahasiswa": Unknown word
6         System.out.print(s: "Masukkan NIM : "); "Masukkan": Unknown word.
7         String nim = scan.nextLine();
8         System.out.print(s: "Masukkan Nama : "); "Masukkan": Unknown word.
9         String nama = scan.nextLine();
10        System.out.print(s: "Masukkan Kelas : "); "Masukkan": Unknown word.
11        String kelas = scan.nextLine(); "kelas": Unknown word.
12        System.out.print(s: "Masukkan IPK : "); "Masukkan": Unknown word.
13        double ipk = Double.parseDouble(scan.nextLine());
14        return new Mahasiswa10(nim, nama, kelas, ipk); "Mahasiswa": Unknown wor
15    }
16
17    Run | Debug | Run main | Debug main
18    public static void main(String[] args) {
19        Scanner scan = new Scanner(System.in); Convert to try-with-resources
20        DoubleLinkedList10 list = new DoubleLinkedList10();
21        int pilihan; "pilihan": Unknown word.
22
23        do {
24            System.out.println(x: "\n===== MENU DOUBLE LINKED LIST =====");
25            System.out.println(x: "1. Tambah data di awal"); "Tambah": Unknown v
26            System.out.println(x: "2. Tambah data di akhir"); "Tambah": Unknown
27            System.out.println(x: "3. Sisipkan data di tengah (setelah NIM)"); '
28            System.out.println(x: "4. Hapus data di awal"); "Hapus": Unknown wor
29            System.out.println(x: "5. Hapus data di akhir"); "Hapus": Unknown we
30            System.out.println(x: "6. Tampilkan data"); "Tampilkan": Unknown wor
31            System.out.println(x: "0. Keluar"); "Keluar": Unknown word.
32            System.out.print(s: "Pilih menu : "); "Pilih": Unknown word.
33            pilihan = Integer.parseInt(scan.nextLine()); "pilihan": Unknown wor
34
35            switch (pilihan) { Convert switch to rule switch
36                case 1:
37                    Mahasiswa10 mhsAwal = inputMahasiswa(scan); "Mahasiswa": Un
38                    list.addFirst(mhsAwal); "Awal": Unknown word.
39                    System.out.println(x: "Data berhasil ditambahkan di awal.");
40                    break;
41                case 2:
42                    Mahasiswa10 mhsAkhir = inputMahasiswa(scan); "Mahasiswa": U
43                    list.addLast(mhsAkhir); "Akhir": Unknown word.
44                    System.out.println(x: "Data berhasil ditambahkan di akhir.");
45                    break;
46                case 3:
47                    System.out.print(s: "Masukkan NIM yang dicari : "); "Masukka
48                    String keyNim = scan.nextLine();
49                    System.out.println(x: "Masukkan data baru: "); "Masukkan": l
50                    Mahasiswa10 dataBaru = inputMahasiswa(scan); "Mahasiswa": U
51                    list.insertAfter(keyNim, dataBaru); "Baru": Unknown word.
52                    break;
53                case 4:
54                    break;
55                case 5:
56                    break;
57                case 6:
58                    list.print();
59                    break;
60                case 0:
61                    System.out.println(x: "Program selesai."); "selesai": Unknow
62                    break;
63                default:
64                    System.out.println(x: "Menu tidak valid."); "tidak": Unknow
65            } while (pilihan != 0); "pilihan": Unknown word.
66
67            scan.close();
68        }
69    }
70}
```

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 2
Masukkan NIM      : 123005
Masukkan Nama     : Harry
Masukkan Kelas    : 1A
Masukkan IPK      : 3.76
Data berhasil ditambahkan di akhir.
```

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru:
Masukkan NIM      : 123010
Masukkan Nama     : Potter
Masukkan Kelas    : 1B
Masukkan IPK      : 3.55
Data berhasil disisipkan setelah NIM 123005
```

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76

NIM      : 123010
Nama     : Potter
Kelas   : 1B
IPK      : 3.55
```

12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Jawab :

Pada Single Linked List, traversal hanya bisa dilakukan dari head ke tail menggunakan pointer next. Untuk mengakses node sebelumnya, harus mengulang traversal dari awal. Pada Double Linked List, setiap node memiliki pointer prev yang menunjuk ke node sebelumnya, sehingga traversal bisa dilakukan dua arah — maju (head → tail) maupun mundur (tail → head).

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Jawab :

Next menyimpan referensi ke node berikutnya dalam linked list. Digunakan saat traversal maju (dari head ke tail), penambahan node di akhir, dan penyisipan node di tengah ke sisi kanan. Prev menyimpan referensi ke node sebelumnya dalam linked list. Digunakan saat traversal mundur (dari tail ke head), penghapusan node (agar node sebelumnya bisa terhubung langsung ke node sesudahnya tanpa iterasi ulang), dan penyisipan node di tengah ke sisi kiri. Keduanya bekerja bersama untuk menjaga keterhubungan dua arah antar node sehingga manipulasi (sisip, hapus) bisa dilakukan secara efisien tanpa harus selalu mulai dari head.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Jawaban :

Konstruktor ini menginisialisasi kondisi awal linked list dalam keadaan kosong. Dengan menetapkan head = null dan tail = null, program menandai bahwa belum ada node sama sekali di dalam list. Ini penting karena method isEmpty() bergantung pada kondisi head == null untuk menentukan apakah list kosong atau tidak, dan setiap operasi penambahan pertama (addFirst/addLast) akan menangani kasus kosong dengan menetapkan head = tail = newNode.

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Jawab :

Ketika linked list hanya memiliki satu node, node tersebut sekaligus merupakan node pertama dan node terakhir. Maka:

head harus menunjuknya karena ia adalah awal list.

tail harus menunjuknya karena ia adalah akhir list.

Jika hanya head yang diset dan tail dibiarkan null, maka operasi yang bergantung pada tail (seperti addLast, removeLast, insertAfter saat current == tail) akan gagal atau melempar NullPointerException. Begitu pula sebaliknya. Keduanya harus menunjuk node yang sama agar list tetap konsisten dan dapat ditelusuri dari kedua arah.

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" Ketika tidak terdapat data pada linked list!

Jawab :

```
public void print() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih kosong.");
        return;
    }
    Node10 current = head;
    while (current != null) {
        current.data.tampil();
        System.out.println();
        current = current.next;
    }
}
```

6. Modifikasi kode program dengan menambahkan method printReverse() untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawab :

```
public void printReverse() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih kosong.");    "masi
        return;
    }
    System.out.println(x: "=== Tampil Terbalik (Tail -> Head) ===");
    Node10 current = tail;
    while (current != null) {
        current.data.tampil();
        System.out.println();
        current = current.prev;
    }
}
```

12.3.2 Verifikasi Hasil Percobaan

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println(x: "Linked List kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Linked List kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
```

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 5
```

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM    : 123005
Nama   : Harry
Kelas : 1A
IPK    : 3.76
```


12.3.3 Pertanyaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:

```
head = head.next;  
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Jawab :

`head = head.next;` Statement ini memajukan pointer head ke node kedua dalam list (node yang sebelumnya ada di posisi `head.next`). Akibatnya, node pertama (yang lama) tidak lagi bisa diakses melalui head, sehingga secara logika sudah "dihapus" dari awal list. Java Garbage Collector kemudian akan membersihkan node tersebut dari memori secara otomatis karena tidak ada referensi yang menunjuknya. `head.prev = null;` Setelah head bergerak ke node kedua, node tersebut masih memiliki pointer prev yang menunjuk ke node lama (yang sudah dihapus). Statement ini memutuskan referensi tersebut dengan menetapkan `head.prev = null`, menjadikan node baru sebagai node pertama yang sesungguhnya (tidak punya pendahulu). Tanpa baris ini, node yang "dihapus" masih bisa diakses via `head.prev`, yang menyebabkan kebocoran memori logis dan perilaku tidak terduga.

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

Jawab :

```
public void removeFirst() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List kosong.");  
        return;  
    }  
    System.out.println(x: "Data berhasil dihapus.");  
    head.data.tampil();  
    System.out.println();  
  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        head = head.next;  
        head.prev = null;  
    }  
}  
  
public void removeLast() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List kosong.");  
        return;  
    }  
    System.out.println(x: "Data berhasil dihapus.");  
    tail.data.tampil();  
    System.out.println();  
  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        tail = tail.prev;  
        tail.next = null;  
    }  
}
```

LINK GITHUB : <https://github.com/nNasq/Jobsheet-12---Double-Linked-List>